Tapwave, Inc. Proprietary

Tapwave®

# Tapwave Advanced Sound API Reference

| Current Version | Date | Author |
|---|---|---|
| REV 0. | 07/24/2003 | MS |
| | | |

## Revision History

| Version | Date | Description | Author |
|---|---|---|---|
| REV 0. | | | |
| REV 0. | | | |
| REV 0. | 7/24/2003 | LIGHT COPY EDIT | MS |

**© Copyright 2003 Tapwave, Inc. All Rights Reserved. Tapwave is a registered trademark of Tapwave, Inc. Palm OS, the Palm logo, Graffiti, HotSync, and PalmSource are registered trademarks of Palm, Inc. Palm, Palm Powered, and the Palm Powered logo are trademarks of Palm, Inc. X-Forge is a trademark of Fathammer, Ltd. Java is a registered trademark of Sun Microsystems, Inc. Windows is a registered trademark of Microsoft Corporation, Inc. All other brands are trademarks or registered trademarks of their respective owners.**

### Application Sound Level

Applications are encouraged to share a common *primary* volume setting across the device. (If you turn the volume up in one app, and then change apps, it should still be louder.)

For compatibility, applications can read (and set) the Game Sound Level in the system preferences. This can be obtained by calling the Palm OS API `PrefGetPreference`. For details, see "Section 43: Preferences" in the *Palm OS Programmer's API Reference at* http://www.palmos.com/dev/support/docs/palmos/Preferences.html. Note that the currently defined sound volume range is 0-64.

However, it is much simpler to use the provided routines for getting and setting the primary volume, and these values are also updated by various volume controls on the device.

Muting is done outside of the main volume control, so that when you unmute the volume returns to the original setting. Tapwave devices will have additional user interface around muting and unmuting, and applications should not need to code the mute state themselves.

#### TwSndGetVolume

**Purpose** Gets the current application volume.

**Prototype**
```
UInt16 TwSndGetVolume(void)
```

**Result** Number between 0 and `sndMaxAmp` (64) indicating the current sound volume.

**Comments** This value is also reflected in the PalmOS sound manager application volume setting. Using this API, the preference value and sound manager value are kept in sync.

**Header** TwSound.h

**Sample** `TBD`

#### TwSndSetVolume

**Purpose** Sets the application volume.

**Prototype**
```
Err TwSndSetVolume(UInt16 newVolume)
```

**Parameters**

`newVolume` A number between 0 and `sndMaxAmp` (64) for the new volume. 0 is effectively muted.

**Result** `sysErrParamErr if the newVolume is out of range.`

**Comments** This value is written to the game sound preference and sets the PalmOS sound manager application global volume setting. The system sounds (clicks, beeps, etc) are further set as a fraction of this level: turning down the application volume also turns down the beeps and clicks.

**Header** `TwSound.h`

**Sample** `TBD`

### TwSndGetBassBoost

**Purpose** Gets the current bass boost level.

**Prototype**
```
UInt16 TwSndGetBassBoost(void)
```

**Result** `Number between 0 and sndMaxAmp (64) indicating the current bass boost level.`

**Comments** This call returns the current bass boost setting. Bass boost applies to all sound played on the device.

**Header** `TwSound.h`

**Sample** `TBD`

### TwSndSetBassBoost

**Purpose** Gets the current application volume.

**Prototype**
```
Err TwSndSetBassBoost(UInt16 boostLevel)
```

**Parameters**

`boostLevel` A number between 0 and `smdMaxAmp` (64) for the new volume. 0 means no bass boost, 64 means maximum bass boost.

**Result** `sysErrParamErr if the boostLevel is out of range.`

**Comments**

**Header** `TwSound.h`

**Sample** `TBD`

### TwSndSetMute

**Purpose** Mute or unmute the device.

**Prototype**
```
void TwSndSetMute(Boolean mute, Uint32 unmuteAt)
```

**Parameters**

`mute` The new setting, true to mute, false to unmute.

`unmuteAt` The time when the mute is canceled. Use zero to mute indefinitely.

**Result**

**Comments** This does not change the result or the behavior of `TwSndGetVolume` and `TwSndSetVolume`. If the volume is changed while muted, the changed value is the new value on unmute.

**Header** `TwSound.h`

**Sample** `TBD`

### TwSndGetMute

**Purpose** Query the mute setting.

**Prototype**
```
Boolean TwSndSetMute(Uint32* unmuteAt)
```

**Parameters**

| | |
|---|---|
| `unmuteAt` | The time when the mute is canceled. Use zero to mute indefinitely. |

**Result** The mute setting – true if muted, false if not.

**Comments**

**Header** `TwSound.h`

**Sample** `TBD`

### TwSndPlaySystemSound

**Purpose** Play a standard system sound.

**Prototype**
```
Err TwSndPlaySystemSound(enum TwSysBeepTag beepID)
```

**Parameters**

| | |
|---|---|
| `beepID` | The ID of the sound to play. An invalid ID returns a `sysErrParamErr`. |

**Result**

**Comments**

**Header** `TwSound.h`

**Sample** `TBD`